

Project clinic I

A question you can answer, data you can actually get, and a baseline by Friday

Fall 2026 · Week 13 · project work begins Thu 12/3; presentations Mon 12/21

What this session is for

You have about two and a half weeks and roughly 31 hours each. The single largest risk to your project is not that the model fails — a negative result, clearly analyzed, is a perfectly good project and the syllabus says so. The risk is spending twelve days on setup and having nothing to say on the thirteenth.

So this handout is about the first three days: choosing a question that is answerable at this scale, getting data that actually loads, and having a working end-to-end result — however bad — before you improve anything.

The one rule. Get to a bad answer fast. A pipeline that runs end to end on ten items and produces a wrong number is worth more on day three than a beautiful half-finished component, because it tells you what the real obstacles are. Everything after that is improvement, and improvement is schedulable in a way that discovery is not.

1 Scoping a question

1.1 The three shapes, and what each needs by day three

Shape	The risk	What you must have by day 3
Extend a paper	the original result does not reproduce	the original result, reproduced on a subset
Improve a model	no measurable axis	a baseline number on your metric
Compare with data	the data never arrives	the data loaded and plotted

Every one of these is a *de-risking* step rather than a result. Do it first, and if it fails you still have eleven days to change course.

1.2 The sentence test

Write your question in this form, filling every blank with something concrete:

“We measure _____ on _____, comparing _____ against _____, and we expect _____.”

If any blank is vague, the project is not yet scoped, and no amount of coding will fix that. Two failure patterns to check for:

- **No comparison.** “We will see whether a CNN represents shape” has no second condition and therefore no possible result. “We compare a CNN’s accuracy on shape-preserving versus texture-preserving distortions” does.
- **No expectation.** If you cannot say what you expect, you will not be able to tell an interesting result from a bug. Write the expectation down *before* you run anything, and keep it — being wrong is a finding, and being wrong without a record is just confusion.

1.3 Scope: what fits in 31 hours

Fits	Does not fit
fine-tuning a pretrained model	training a large model from scratch
a few thousand images	ImageNet
2–4 conditions	a factorial sweep
one dataset, done carefully	three datasets, done carelessly
a laptop or Colab’s free tier	anything needing a reserved GPU

Assume the network is slow. Downloads in this course have run at kilobytes per second. Start every download on day one, in the background, and build against a small local subset while it runs. A project blocked on a download at day ten is the most avoidable failure there is.

2 Getting data

2.1 Where to look, in order

1. **Datasets you already have.** Assignments 1–5 shipped stimulus sets, activation matrices, and real neural data. Reusing one is not cheating; it is good sense, and it means day one is a loading step rather than a search.
2. **The paper’s own release.** Many papers post data on OSF, Zenodo, or GitHub. Check for a “Data availability” statement before you write to anyone.
3. **Standard repositories** — OpenNeuro for neuroimaging, CRCNS for electrophysiology, the Natural Scenes Dataset, Brain-Score’s public benchmarks, HuggingFace Datasets for text and images.
4. **Ask.** Authors often share on request, but budget a week for a reply and never make the project depend on one arriving.

2.2 Load it, then look at it

Before any analysis, and without exception:

```

print(X.shape, X.dtype)
print(np.isnan(X).sum(), np.isinf(X).sum())
print(X.min(), X.max(), X.mean())
print(np.unique(y, return_counts=True)) # class balance
plt.imshow(X[0]); plt.show() # LOOK at one item

```

Five lines, and they catch: transposed dimensions, images in $[0, 255]$ where the model expects $[0, 1]$, a class that occurs eleven times, dead units that will produce nan correlations, and files that decoded into noise. Every one of those has cost somebody a week.

2.3 Write a loader, once

```

def load_data(root, subset=None):
    """Return X, y, meta. The ONLY place that touches the filesystem."""
    ...
    return X, y, meta

```

Everything downstream calls this. When you discover on day nine that the labels were off by one, you fix it in one place and rerun — rather than hunting for six notebooks that each did their own loading slightly differently.

3 The end-to-end skeleton

Write this on day two, with every step deliberately stupid, and run it on ten items.

```

X, y, meta = load_data(ROOT, subset=10) # 1. ten items
model = get_model() # 2. pretrained, off the shelf
preds = predict(model, X) # 3. no batching, no speed
score = evaluate(preds, y) # 4. one number
plot(score) # 5. one figure

```

Why ten items. Because it runs in seconds, so you can iterate; because you can verify every number by hand; and because scaling from 10 to 10,000 is a known, boring change, whereas scaling from “nothing runs” to “something runs” is not. When the skeleton works, raise subset and go and have lunch.

3.1 Two baselines you always need

Chance. What score does a model that knows nothing get? For a classifier with imbalanced classes it is the majority proportion, not $1/k$. Compute it, do not assume it.

A trivial model. Whatever the sophisticated thing is, run the dumb thing too — raw pixels instead of network features, logistic regression instead of a fine-tuned transformer, mean luminance instead of a shape descriptor. If the dumb thing does nearly as well, you have

learned the most important fact about your problem, and you have learned it on day three rather than in the last slide of your talk.

4 Reproducibility, cheaply

Four habits, each costing under a minute, that will save your group hours:

1. **One seed, set once, at the top.** `np.random.default_rng(0)`, `torch.manual_seed(0)`. And remember Recitation 3: one seed makes it reproducible, not meaningful. For any headline number, run several.
2. **Save results, not just figures.** Write the numbers to a `.csv` or `.npz` as you go. Regenerating a figure from a saved array takes ten seconds; regenerating it from scratch takes an hour and may not match.
3. **Never edit a figure by hand.** If it needs a change, change the code. Hand-edited figures cannot be regenerated, and you will need to regenerate them.
4. **One notebook per question, not one per person.** Three people working in three copies of the same notebook is the most common way a group project loses a day to merge conflicts. Split by *question*, with a shared `utils.py` that all three import.

5 A schedule that works

By	You have
day 3	data loaded and looked at; the skeleton runs on 10 items; chance and a trivial baseline
day 6	the real comparison run once, at full size, with one figure
day 9	the comparison run across seeds, with error bars; one control
day 12	every figure final; the result stated in one sentence
day 14	slides built and rehearsed once, out loud, with a timer

The deadline that matters is day 9, not day 14. If the main comparison is not running by day 9 you do not have time to add error bars, and a result without error bars cannot be defended — which is the single most common way a technically fine project loses marks.

6 Ten questions to ask your own project

1. What is the comparison? If there is only one condition, there is no result.
2. What did you *expect*, and did you write it down first?
3. What is chance, and did you compute it or assume it?
4. What does the trivial baseline get?

5. How many seeds, and what is the spread?
6. What is the noise ceiling, if you are comparing against data?
7. What single change would most likely overturn your conclusion?
8. Did you touch the test set more than once?
9. Can someone else run your pipeline from your loader?
10. If the result had come out the other way, would the project still work?

Question 10 is the one to ask on day one. If the answer is no, the project is a bet rather than an experiment, and you have two weeks.